

Fast Single-Point Evaluation of Univariate Polynomials in the Boolean-Kernel Basis

Ali Mkhida
Algorizk Labs

October 5, 2026

Abstract

We study single-point evaluation of large univariate polynomials represented in a Boolean-kernel basis and compare it with conventional univariate representations relevant to zero-knowledge proof systems. For $N = 2^m$, the basis is indexed by $y \in \{0, 1\}^m$ and defined by

$$K_y(X) = \prod_{i=0}^{m-1} (X^{2^i} + y_i).$$

Its structure gives a binary evaluation tree whose elementary fold is

$$(a, b) \mapsto t(a + b) + b, \quad t = x^{2^i}.$$

We implement and optimize this evaluator over the Goldilocks field. For $N = 2^{20}$ on an Apple M1 processor, the final same-run cross-basis benchmark gives a central estimate of 1.0560 ms for the optimized Boolean-kernel evaluator and 1.4116 ms for the parallel blocked monomial baseline. This corresponds to a $1.337\times$ evaluation-speed advantage, or approximately 25.2% lower wall-clock time. A separate optimization-only run reached $892.57\ \mu\text{s}$, illustrating the observed run-to-run variation. We describe the evaluation algorithm, its exact arithmetic cost, the implementation optimizations responsible for the measured speedup, and the limitations of transferring this primitive-level gain to complete proof systems.

1 Introduction

Polynomial arithmetic is a central computational component of modern zero-knowledge proof systems. Provers manipulate large polynomials during arithmetization, constraint evaluation, low-degree testing, polynomial commitment, quotient construction, and opening procedures. Transparent proof systems based on Reed–Solomon proximity testing, including FRI-based constructions and STARKs, make polynomial evaluation and encoding particularly visible components of the prover computation [BSBHR18].

The representation chosen for a polynomial determines the cost of many of these operations. A polynomial may be represented by its monomial coefficients, by its values on a structured evaluation domain, or by coefficients in another basis adapted to a particular algebraic operation. These representations describe the same polynomial space but expose different arithmetic structure and different opportunities for parallelism.

This work isolates one basic operation:

Given a degree- $< N$ univariate polynomial P and an arbitrary point x in the base field, compute $P(x)$ from the chosen representation.

We deliberately study this primitive independently of a complete proof system. This allows the effect of the representation itself to be measured under a common field implementation, machine, input size, and benchmarking methodology.

Let

$$N = 2^m.$$

In addition to the monomial representation and a roots-of-unity Lagrange/evaluation representation, we study the following family of univariate polynomials:

$$K_y(X) = \prod_{i=0}^{m-1} (X^{2^i} + y_i), \quad y = (y_0, \dots, y_{m-1}) \in \{0, 1\}^m.$$

We refer to this family as the *Boolean-kernel basis*. A polynomial represented in this basis is written

$$P(X) = \sum_{y \in \{0,1\}^m} c_y K_y(X).$$

The basis admits a simple binary reduction rule. Fix an evaluation point x , and define

$$t_i = x^{2^i}.$$

At level i , two coefficients a and b whose indices differ only in bit i are replaced by

$$F_{t_i}(a, b) = t_i(a + b) + b.$$

Applying this rule recursively reduces N coefficients to the single value $P(x)$. The full tree contains exactly $N - 1$ folds, and all folds at the same level are independent.

This structure makes the evaluation algorithm both linear in the number of input coefficients and naturally parallel. Since an arbitrary coefficient vector must in general be inspected in full, an $O(N)$ algorithm already has optimal asymptotic input complexity for this task. The relevant question is therefore whether the basis gives a practically favorable constant factor and memory-access pattern.

We answer this experimentally using a Rust implementation over the Goldilocks field

$$\mathbb{F}_p, \quad p = 2^{64} - 2^{32} + 1.$$

The optimized implementation combines coarse-grained subtree parallelism, a specialized Goldilocks reduction, a fused kernel-fold operation, bounds-check-free inner loops, direct first-level reduction, and worker-local reusable scratch storage.

For

$$N = 2^{20} = 1,048,576,$$

the final same-run cross-basis benchmark on the tested Apple M1 machine gives a central estimate of

$$\boxed{1.0560 \text{ ms}}$$

for the optimized Boolean-kernel evaluator and

$$1.4116 \text{ ms}$$

for the blocked parallel monomial evaluator. Thus the kernel evaluator is

$$\boxed{1.337\times}$$

faster in this controlled comparison, corresponding to approximately

$$\boxed{25.2\%}$$

less wall-clock evaluation time. During kernel-only optimization, a separate run reached $892.57 \mu\text{s}$; we report that value as a best observed implementation result but do not mix it with competitor measurements from another run when computing cross-basis speedups.

This comparison is intentionally an operation-level result. It does *not* imply a $1.337\times$ speedup for a complete STARK, FRI prover, or polynomial commitment scheme. A complete protocol also contains encoding, low-degree extension, hashing, commitment, transcript, query, and other polynomial operations, and a change of representation may create conversion costs elsewhere. The result instead establishes that representation alone can materially change the cost of arbitrary-point evaluation and motivates studying whether this basis can be maintained across a larger fraction of a prover pipeline.

Related work on transparent proof systems already demonstrates the importance of linear- and quasi-linear-time polynomial machinery [BSBHR18]. Other polynomial commitment constructions have also highlighted the cost of moving between polynomial representations; for example, Zeromorph explicitly addresses interactions between multilinear polynomials and univariate commitment machinery [KT23]. Our scope is different: we experimentally isolate a univariate basis and measure its effect on a concrete polynomial operation.

Contributions. The contributions of this work are:

1. We give a direct binary-fold evaluation algorithm for the univariate Boolean-kernel basis and derive its exact arithmetic cost.
2. We provide an optimized Rust implementation over the Goldilocks field with correctness tests and reproducible benchmarks.
3. We compare single-point evaluation against conventional univariate representations under a common implementation methodology, including a parallel monomial baseline rather than an artificially serialized competitor.
4. We provide an optimization ablation covering arithmetic reduction, memory organization, bounds checks, scratch-buffer reuse, subtree size, and unsuccessful tree-fusion strategies.
5. We distinguish primitive-level speedups from end-to-end proof-system claims and identify the representation-conversion and integration questions that must be resolved before transferring the result to a complete prover.

The remainder of the paper first defines the polynomial representations formally, then derives the Boolean-kernel evaluation algorithm, describes the implementation and benchmark methodology, reports the experimental results, and discusses implications for proof-system design.

2 Polynomial Representations

This section fixes the notation used throughout the paper and defines the three univariate representations compared experimentally.

2.1 Ambient polynomial space

Let $\mathbb{F}$ be a field and let

$$N = 2^m$$

for an integer $m \geq 1$. We work in the N -dimensional $\mathbb{F}$ -vector space

$$\mathbb{F}[X]_{<N} = \{P \in \mathbb{F}[X] : \deg P < N\}.$$

Every representation considered below encodes exactly one polynomial in this space using N field elements.

For an integer

$$e = \sum_{i=0}^{m-1} e_i 2^i, \quad e_i \in \{0, 1\},$$

we write

$$\text{bits}(e) = (e_0, \dots, e_{m-1}).$$

For Boolean vectors $u, v \in \{0, 1\}^m$, we write

$$u \subseteq v$$

when $u_i \leq v_i$ for every i . Equivalently, the support of u is a subset of the support of v .

2.2 Monomial representation

The standard monomial basis of $\mathbb{F}[X]_{<N}$ is

$$1, X, X^2, \dots, X^{N-1}.$$

A polynomial is represented by coefficients

$$a_0, \dots, a_{N-1} \in \mathbb{F}$$

such that

$$P(X) = \sum_{e=0}^{N-1} a_e X^e.$$

For a point $x \in \mathbb{F}$, Horner evaluation computes

$$P(x) = a_0 + x(a_1 + x(a_2 + \dots + xa_{N-1})),$$

or equivalently by processing the coefficients in reverse order with the recurrence

$$r_{j+1} = xr_j + a_{N-1-j}, \quad r_0 = 0.$$

The serial dependency chain makes ordinary Horner evaluation inherently sequential, although a long coefficient vector can be partitioned into blocks and the blocks evaluated in parallel. Our experimental comparison therefore includes both serial Horner evaluation and a blocked parallel monomial baseline.

2.3 Roots-of-unity Lagrange representation

Assume that $\mathbb{F}$ contains a primitive N -th root of unity ω . Define the multiplicative evaluation domain

$$H = \{1, \omega, \omega^2, \dots, \omega^{N-1}\}.$$

A polynomial $P \in \mathbb{F}[X]_{<N}$ can be represented by its values

$$v_j = P(\omega^j), \quad 0 \leq j < N.$$

The associated Lagrange basis consists of polynomials

$$L_j(X) = \prod_{\substack{0 \leq k < N \\ k \neq j}} \frac{X - \omega^k}{\omega^j - \omega^k},$$

which satisfy

$$L_j(\omega^k) = \begin{cases} 1, & j = k, \\ 0, & j \neq k. \end{cases}$$

Hence

$$P(X) = \sum_{j=0}^{N-1} v_j L_j(X).$$

For a roots-of-unity domain one may also use

$$L_j(X) = \frac{X^N - 1}{N(X - \omega^j)} \omega^j,$$

provided $X \neq \omega^j$. This identity is useful for arbitrary-point evaluation because all denominators $x - \omega^j$ can be inverted with a single batch inversion.

This representation is especially natural when a proof system already stores polynomial values on an FFT/NTT domain. Our benchmark measures arbitrary-point evaluation from this representation rather than the separate cost of constructing the representation.

2.4 Boolean-kernel representation

For each Boolean vector

$$y = (y_0, \dots, y_{m-1}) \in \{0, 1\}^m,$$

define

$$K_y(X) = \prod_{i=0}^{m-1} (X^{2^i} + y_i).$$

There are exactly $2^m = N$ such polynomials.

A Boolean-kernel coefficient vector is a family

$$(c_y)_{y \in \{0,1\}^m}$$

representing the polynomial

$$P(X) = \sum_{y \in \{0,1\}^m} c_y K_y(X).$$

Although every K_y has degree at most $N - 1$, and in fact every K_y contains the top monomial X^{N-1} , the family is nevertheless a basis. Its coefficient matrix is a Boolean zeta matrix after a reversal of the monomial index.

Lemma 1 (Monomial coefficients of K_y). *Let*

$$e = \sum_{i=0}^{m-1} e_i 2^i$$

with $e_i \in \{0, 1\}$, and define the Boolean complement vector

$$\bar{e} = (1 - e_0, \dots, 1 - e_{m-1}).$$

Then

$$[X^e]K_y(X) = \begin{cases} 1, & \bar{e} \subseteq y, \\ 0, & \text{otherwise.} \end{cases}$$

Proof. Expanding

$$K_y(X) = \prod_{i=0}^{m-1} (X^{2^i} + y_i),$$

the monomial X^e can arise in exactly one way because the powers $2^0, \dots, 2^{m-1}$ give the unique binary expansion of e .

For each index i with $e_i = 1$, the term X^{2^i} must be selected. For each index i with $e_i = 0$, the constant term y_i must be selected. Therefore

$$[X^e]K_y(X) = \prod_{i:e_i=0} y_i.$$

Since every y_i is either 0 or 1, this product equals 1 exactly when $y_i = 1$ for every i such that $e_i = 0$. This condition is precisely

$$\bar{e} \subseteq y.$$

□

Theorem 2 (Boolean-kernel basis). *The family*

$$\mathcal{K}_m = \{K_y(X) : y \in \{0, 1\}^m\}$$

is a basis of $\mathbb{F}[X]_{<N}$.

Proof. There are exactly N polynomials in $\mathcal{K}_m$, so it is enough to prove linear independence.

Index the monomial rows not directly by e , but by the Boolean vector

$$z = \bar{e}.$$

By 1, the coefficient matrix A of the family $\mathcal{K}_m$ in the monomial basis is

$$A_{z,y} = \mathbf{1}[z \subseteq y].$$

This is the zeta matrix of the Boolean subset lattice.

Order the Boolean vectors by nondecreasing Hamming weight, breaking ties arbitrarily. Whenever $z \subseteq y$, either $z = y$ or

$$|z| < |y|.$$

Consequently, under this ordering, A is upper triangular with diagonal entries

$$A_{y,y} = 1.$$

Hence

$$\det A = 1 \neq 0$$

in every field, so A is invertible. Therefore the polynomials K_y are linearly independent and form a basis of $\mathbb{F}[X]_{<N}$. □

2.5 Relation to the Boolean zeta transform

The proof above also gives an explicit relationship between monomial and Boolean-kernel coefficients.

Let

$$P(X) = \sum_{e=0}^{N-1} a_e X^e = \sum_{y \in \{0,1\}^m} c_y K_y(X).$$

For the exponent whose Boolean complement is z , that is

$$e = (N - 1) \text{ xor } z,$$

1 gives

$$a_e = \sum_{y: z \subseteq y} c_y.$$

Thus, after complementing the monomial index, the map from kernel coefficients to monomial coefficients is the Boolean superset-zeta transform.

The inverse map is Boolean Möbius inversion:

$$c_y = \sum_{z: y \subseteq z} (-1)^{|z|-|y|} a_{(N-1) \text{ xor } z}.$$

Algorithmically, both transforms can be implemented in

$$O(N \log N)$$

field operations by the usual butterfly-style zeta/Möbius transform.

This conversion cost is deliberately separated from the single-point evaluation benchmarks. In each benchmark the polynomial is assumed to already be available in the representation being measured. Conversion costs become important when considering an end-to-end proof-system pipeline and are discussed separately later in the paper.

2.6 A small example

Take $m = 2$, so $N = 4$. The four kernel polynomials are

$$\begin{aligned} K_{00}(X) &= X(X^2) = X^3, \\ K_{01}(X) &= (X + 1)X^2 = X^3 + X^2, \\ K_{10}(X) &= X(X^2 + 1) = X^3 + X, \\ K_{11}(X) &= (X + 1)(X^2 + 1) = X^3 + X^2 + X + 1. \end{aligned}$$

Here the first Boolean coordinate corresponds to the factor $X^{2^0} + y_0$, and the second to $X^{2^1} + y_1$.

The coefficient pattern is triangular after complementing the monomial index. In particular, the constant monomial occurs only in K_{11} , the monomial X occurs in K_{01} and K_{11} , the monomial X^2 occurs in K_{10} and K_{11} , and X^3 occurs in all four kernel polynomials.

This subset structure is the algebraic reason that the representation is invertible. In the next section we show that the same product structure also produces a particularly simple binary evaluation algorithm.

3 Boolean-Kernel Evaluation

We now derive the single-point evaluation algorithm for the Boolean-kernel representation introduced in 2.

Let

$$P(X) = \sum_{y \in \{0,1\}^m} c_y \prod_{i=0}^{m-1} (X^{2^i} + y_i),$$

and fix an evaluation point

$$x \in \mathbb{F}.$$

Define

$$t_i = x^{2^i}, \quad 0 \leq i < m.$$

The key observation is that two basis polynomials whose indices differ only in one Boolean coordinate can be combined locally.

3.1 The elementary fold identity

Fix a level i . Let

$$u = (u_0, \dots, u_{i-1}, u_{i+1}, \dots, u_{m-1}) \in \{0, 1\}^{m-1}$$

denote the common values of all Boolean coordinates other than i . Let

$$K_{u,0}(X)$$

and

$$K_{u,1}(X)$$

denote the two kernel polynomials whose i -th coordinate is respectively 0 and 1.

By definition,

$$K_{u,0}(X) = X^{2^i} \prod_{j \neq i} (X^{2^j} + u_j),$$

while

$$K_{u,1}(X) = (X^{2^i} + 1) \prod_{j \neq i} (X^{2^j} + u_j).$$

Suppose the corresponding coefficients are

$$a, b \in \mathbb{F}.$$

Their contribution to $P(x)$ is

$$aK_{u,0}(x) + bK_{u,1}(x).$$

Factoring the common part gives

$$\begin{aligned} aK_{u,0}(x) + bK_{u,1}(x) &= (ax^{2^i} + b(x^{2^i} + 1)) \prod_{j \neq i} (x^{2^j} + u_j) \\ &= (x^{2^i}(a+b) + b) \prod_{j \neq i} (x^{2^j} + u_j). \end{aligned}$$

Therefore, after eliminating Boolean coordinate i , the two coefficients a and b are replaced by the single coefficient

$$\boxed{F_{t_i}(a, b) = t_i(a+b) + b}$$

with

$$t_i = x^{2^i}.$$

Lemma 3 (Pair-fold identity). *Let $a, b \in \mathbb{F}$, let $i \in \{0, \dots, m-1\}$, and let u fix all Boolean coordinates other than i . Then*

$$aK_{u,0}(x) + bK_{u,1}(x) = F_{t_i}(a, b) \prod_{j \neq i} (x^{2^j} + u_j),$$

where

$$F_{t_i}(a, b) = t_i(a+b) + b.$$

Proof. The identity follows directly from the factorization above. □

3.2 Recursive reduction

The pair-fold identity can be applied recursively over all Boolean coordinates. We use the coefficient ordering in which the integer index

$$y = \sum_{i=0}^{m-1} y_i 2^i$$

stores the Boolean vector

$$(y_0, \dots, y_{m-1}).$$

Hence the least significant bit corresponds to the first folding level.

Let

$$v_j^{(0)} = c_j, \quad 0 \leq j < N.$$

After level i , the active vector has length

$$N_i = \frac{N}{2^i}.$$

For

$$0 \leq j < \frac{N_i}{2},$$

define

$$v_j^{(i+1)} = F_{t_i} \left(v_{2j}^{(i)}, v_{2j+1}^{(i)} \right).$$

Equivalently,

$$v_j^{(i+1)} = t_i \left(v_{2j}^{(i)} + v_{2j+1}^{(i)} \right) + v_{2j+1}^{(i)}.$$

After m levels only one field element remains.

Theorem 4 (Correctness of Boolean-kernel evaluation). *Let*

$$P(X) = \sum_{y \in \{0,1\}^m} c_y K_y(X).$$

Construct the vectors

$$v^{(0)}, v^{(1)}, \dots, v^{(m)}$$

by the recurrence above using

$$t_i = x^{2^i}.$$

Then

$$v_0^{(m)} = P(x).$$

Proof. We prove a stronger invariant.

After completing levels

$$0, 1, \dots, i-1,$$

the vector $v^{(i)}$ is indexed by the remaining Boolean coordinates

$$(y_i, \dots, y_{m-1}).$$

For every remaining index

$$z = (z_i, \dots, z_{m-1}) \in \{0,1\}^{m-i},$$

the value $v_z^{(i)}$ is the coefficient of

$$\prod_{j=i}^{m-1} (X^{2^j} + z_j)$$

after the already-eliminated coordinates

$$0, \dots, i-1$$

have been evaluated at x .

The invariant is true for $i = 0$ by the original representation of P .

Assume it holds for some $i < m$. Pair entries whose remaining indices agree in every coordinate except y_i . By 3, replacing such a pair

$$(a, b)$$

by

$$F_{t_i}(a, b)$$

exactly evaluates the factor associated with coordinate i , while leaving the factors for coordinates

$$i+1, \dots, m-1$$

unchanged. Therefore the invariant holds for $i+1$.

After m levels no Boolean coordinates remain, so the unique surviving value is exactly

$$P(x).$$

□

3.3 Exact arithmetic complexity

At level i , the active vector has length

$$\frac{N}{2^i},$$

so the number of folds is

$$\frac{N}{2^{i+1}}.$$

Summing over all m levels gives

$$\sum_{i=0}^{m-1} \frac{N}{2^{i+1}} = N \sum_{k=1}^m 2^{-k} = N \left(1 - \frac{1}{2^m}\right) = N - 1.$$

Therefore the evaluation tree contains exactly

$$\boxed{N - 1}$$

folds.

Each fold

$$F_t(a, b) = t(a + b) + b$$

contains, at the field-operation level,

$$1$$

multiplication and

$$2$$

additions.

Hence the tree itself requires exactly

$$\boxed{N - 1}$$

field multiplications and

$$\boxed{2(N-1)}$$

field additions.

The level parameters are generated by

$$t_0 = x, \quad t_{i+1} = t_i^2.$$

Thus producing all

$$t_0, \dots, t_{m-1}$$

requires

$$\boxed{m-1}$$

field squarings.

We summarize the operation count in the following proposition.

Proposition 5 (Exact operation count). *For $N = 2^m$, Boolean-kernel single-point evaluation requires*

$$N-1$$

kernel folds. Using the direct fold formula, this corresponds to

$$N-1$$

field multiplications,

$$2(N-1)$$

field additions, and

$$m-1$$

field squarings for generation of the level parameters t_i .

3.4 Parallel structure

The dependency graph is a complete binary reduction tree. At each fixed level i , all folds operate on disjoint coefficient pairs and are therefore independent.

The number of available independent folds at level i is

$$\frac{N}{2^{i+1}}.$$

Thus the lower levels expose abundant data parallelism, while the final levels contain progressively less work.

A naive parallel implementation may synchronize globally after every tree level. For large N , however, such synchronization can dominate the shrinking amount of useful work near the top of the tree. The optimized implementation studied later therefore groups multiple consecutive levels inside independent power-of-two subtrees and parallelizes only across those larger subtrees.

This changes the scheduling strategy but not the arithmetic graph or the number of folds.

3.5 Input complexity

The evaluation algorithm uses

$$O(N)$$

field operations and reads an arbitrary vector of N coefficients.

For the unrestricted problem considered here, this linear dependence on the input length is unavoidable in the worst case. To see this, suppose an algorithm does not inspect some coefficient c_y . For any evaluation point x such that

$$K_y(x) \neq 0,$$

changing only c_y changes

$$P(x)$$

while leaving all inspected coefficients unchanged. Hence an algorithm that is correct on arbitrary coefficient vectors must in general obtain information about every coefficient.

Accordingly, the principal optimization target is not the asymptotic complexity but the constant factor: field reduction, memory traffic, allocation, bounds checking, scheduling, and the organization of the fold tree.

3.6 Example for $N = 8$

Let

$$m = 3, \quad N = 8.$$

The input coefficient vector is

$$(c_0, c_1, c_2, c_3, c_4, c_5, c_6, c_7).$$

Set

$$t_0 = x, \quad t_1 = x^2, \quad t_2 = x^4.$$

The first level computes

$$\begin{aligned} u_0 &= F_{t_0}(c_0, c_1), \\ u_1 &= F_{t_0}(c_2, c_3), \\ u_2 &= F_{t_0}(c_4, c_5), \\ u_3 &= F_{t_0}(c_6, c_7). \end{aligned}$$

The second level computes

$$\begin{aligned} w_0 &= F_{t_1}(u_0, u_1), \\ w_1 &= F_{t_1}(u_2, u_3). \end{aligned}$$

Finally,

$$P(x) = F_{t_2}(w_0, w_1).$$

The tree uses

$$4 + 2 + 1 = 7 = N - 1$$

folds, illustrating the general count of 5.

4 Implementation and Optimization

This section describes the Rust implementation used in the experiments and the sequence of optimizations that led to the final evaluator. The goal is not merely to present the fastest implementation, but to separate algorithmic structure from low-level engineering effects and to document which optimization ideas did and did not improve performance on the tested Apple M1 platform.

4.1 Implementation setting

The implementation is written in Rust and uses Rayon for CPU parallelism. All basis implementations in a given benchmark use the same finite-field backend.

The experiments in this paper use the Goldilocks field

$$\mathbb{F}_p, \quad p = 2^{64} - 2^{32} + 1.$$

Field elements are stored canonically in a single 64-bit machine word.

The optimized Boolean-kernel evaluator operates on coefficient vectors of length

$$N = 2^m.$$

The mathematical fold is

$$F_t(a, b) = t(a + b) + b.$$

The implementation therefore has two distinct optimization layers:

1. optimize the field-level realization of F_t ;
2. organize the binary tree so that memory traffic, allocation, and synchronization are minimized.

4.2 Baseline serial implementation

The direct serial implementation copies the coefficient vector into a work buffer and performs the tree level by level.

If the active prefix has length L , one level computes

$$\frac{L}{2}$$

folds

$$\text{buf}[j] \leftarrow F_t(\text{buf}[2j], \text{buf}[2j+1]),$$

then replaces L by $L/2$.

This implementation closely follows the recurrence of 3 and is used as a correctness reference.

4.3 Coarse-grained subtree parallelism

A first parallel implementation may assign all independent folds at a tree level to Rayon and synchronize before proceeding to the next level. This exposes maximal parallelism at the bottom of the tree, but it also creates a global synchronization point at every level.

Instead, the optimized implementation decomposes the coefficient vector into contiguous power-of-two blocks of size

$$B.$$

Each block represents a complete local subtree. Rayon distributes those subtrees across workers, and each worker reduces an entire subtree locally before returning its root.

The remaining roots form a much smaller top tree, which is finished serially.

For

$$N = 2^{20}, \quad B = 4096 = 2^{12},$$

the input is split into

$$\frac{2^{20}}{2^{12}} = 256$$

independent local subtrees.

This organization removes the need for a global barrier after each of the first twelve tree levels.

4.4 Fast Goldilocks multiplication

A generic implementation may multiply two canonical 64-bit residues into a 128-bit product and reduce the result modulo

$$p = 2^{64} - 2^{32} + 1.$$

The Goldilocks modulus permits a more specialized reduction. Write

$$x = \ell + 2^{64}h,$$

where

$$0 \leq \ell, h < 2^{64}.$$

Let

$$\varepsilon = 2^{32} - 1.$$

Since

$$2^{64} \equiv \varepsilon \pmod{p},$$

we have

$$x \equiv \ell + h\varepsilon \pmod{p}.$$

Split

$$h = h_{\text{lo}} + 2^{32}h_{\text{hi}},$$

with

$$0 \leq h_{\text{lo}}, h_{\text{hi}} < 2^{32}.$$

Using

$$2^{32}\varepsilon = 2^{64} - 2^{32} \equiv -1 \pmod{p},$$

we obtain

$$h\varepsilon = h_{\text{lo}}\varepsilon + 2^{32}h_{\text{hi}}\varepsilon \equiv h_{\text{lo}}\varepsilon - h_{\text{hi}} \pmod{p}.$$

Therefore

$$\boxed{x \equiv \ell - h_{\text{hi}} + h_{\text{lo}}(2^{32} - 1) \pmod{p}.$$

This form avoids the repeated high-half multiplications generated by the more generic reduction routine.

On the tested Apple M1 system, the multiplication microbenchmark improved from approximately

$$2.2372 \text{ ns}$$

to

$$2.0559 \text{ ns},$$

an improvement of roughly

$$8\%.$$

4.5 Fused kernel fold

The direct fold

$$F_t(a, b) = t(a + b) + b$$

could be implemented as one modular addition, one modular multiplication, and a second modular addition.

The optimized implementation instead fuses the final $+b$ into the 128-bit product before Goldilocks reduction.

Let

$$s = (a + b) \bmod p.$$

The implementation forms the full product

$$ts$$

as a 128-bit value, adds b to that 128-bit value with carry, and only then applies the fast Goldilocks reduction.

This preserves the mathematical value

$$t(a + b) + b \pmod{p}$$

while avoiding a second complete modular-addition sequence after the multiplication.

The isolated fold microbenchmark improved from approximately

$$4.0591 \text{ ns}$$

to

$$3.3585 \text{ ns},$$

corresponding to an improvement of about

$$17.3\%.$$

4.6 Bounds-check elimination

After arithmetic optimization, the inner tree loop became short enough that Rust bounds checks were visible in the generated assembly.

The safe indexed loop

$$\text{buf}[2j], \quad \text{buf}[2j+1], \quad \text{buf}[j]$$

generated two conditional bounds-check branches per fold iteration on the tested compiler target.

The optimized implementation uses a small audited helper based on raw pointers. For an active length L , the loop is executed only for

$$0 \leq j < \frac{L}{2}.$$

Therefore

$$2j + 1 < L$$

and

$$j < \frac{L}{2} \leq L.$$

The output location of iteration j also cannot overwrite a future input location, because for every future index $k > j$,

$$2k > j.$$

Thus the in-place access pattern is valid, and the unchecked helper removes the per-iteration bounds checks while preserving the same sequential semantics.

For a 4096-element level, the measured runtime improved from

$$6.9671 \mu\text{s}$$

to

$$6.3575 \mu\text{s},$$

an improvement of approximately

$$8.75\%.$$

4.7 Direct first-level reduction

The first subtree implementation began each local job by copying all B input coefficients into a private mutable buffer:

$$B \longrightarrow B.$$

It then applied the first tree level:

$$B \longrightarrow \frac{B}{2}.$$

The optimized implementation eliminates this full copy. It reads pairs directly from the immutable input slice and writes the first-level outputs into a newly allocated half-size buffer:

$$\boxed{B \longrightarrow \frac{B}{2}.}$$

For

$$B = 4096,$$

this changes the local scratch allocation from 4096 field elements to 2048 field elements and eliminates the initial full-block copy.

At the first-stage microbenchmark level, the measured runtime improved from

$$6.9031 \mu\text{s}$$

to

$$6.1571 \mu\text{s},$$

an improvement of approximately

$$10.8\%.$$

At the full-evaluator level, this optimization reduced the best measured runtime from approximately

$$1.0179 \text{ ms}$$

to

$$0.93533 \text{ ms}.$$

4.8 Worker-local scratch reuse

Even after direct first-level reduction, creating a new half-size vector for every subtree still incurs repeated allocation overhead.

The final implementation uses Rayon worker-local state. Each worker owns a reusable scratch vector and processes multiple subtrees with that same allocation.

This differs from an earlier unsuccessful global-buffer strategy: the source coefficient array remains immutable and all workers still read their assigned input subtrees directly in parallel. Only the private scratch allocation is reused.

In the worker-scratch optimization experiment, the same-run baseline required

$$945.77 \mu\text{s},$$

whereas worker-local scratch reuse required

$$892.57 \mu\text{s}.$$

This is the best observed kernel-only optimization run; the final cross-basis benchmark is reported separately in 6. This corresponds to an improvement of approximately

$$5.6\%$$

in that run.

The resulting throughput was approximately

$$1.175 \text{ Gelem/s}.$$

4.9 Subtree-size tuning

The subtree size B trades scheduling overhead against local work size and available parallelism.

After the arithmetic and memory optimizations above, we re-ran the block size sweep because the optimum found for an earlier, slower implementation need not remain optimal.

A broad sweep over

$$B \in \{1024, 2048, 4096, 8192, 16384\}$$

identified

$$2048, 4096, 8192$$

as the competitive region.

A longer controlled comparison gave:

Block size B	Median runtime
2048	1.0151 ms
4096	920.45 μ s
8192	917.46 μ s

The difference between 4096 and 8192 was only about

$$0.32\%,$$

which is inside the observed run-to-run noise. We therefore retain

$$\boxed{B = 4096}$$

as the conservative default.

4.10 Optimization progression

The approximate progression of the best subtree-parallel implementation at

$$N = 2^{20}$$

was

$$1.2530 \text{ ms} \rightarrow 1.2007 \text{ ms} \rightarrow 1.1619 \text{ ms} \rightarrow 1.0179 \text{ ms} \rightarrow 0.93533 \text{ ms} \rightarrow \boxed{0.89257 \text{ ms}}.$$

Relative to the earlier 1.2530-ms subtree baseline, the final observed runtime is lower by approximately

$$28.8\%.$$

The sequence is important: no single optimization accounts for the full gain. Arithmetic, compiler-generated checks, memory traffic, and allocation all become visible as preceding bottlenecks are removed.

4.11 Rejected optimization attempts

Several plausible optimizations did not improve performance and were not included in the final evaluator.

Global shared-buffer reuse. A version that copied the complete coefficient vector into one shared mutable buffer and then reduced disjoint chunks avoided per-subtree allocation, but the initial global copy serialized too much memory traffic. It was slower than the local-subtree design.

Manual four-way loop unrolling. Explicitly computing four independent folds per source loop iteration did not improve the generated scalar code enough to offset the additional register pressure and code size.

Alternative modular addition. A manually derived Goldilocks addition sequence was compared with the compiler-generated implementation. The compiler’s existing sequence was already shorter, so the alternative was rejected before integration.

Three-level tree fusion. A local $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ fused tree reduces intermediate memory traffic, but it also increases dependency depth and register pressure. The fused version was slower than the ordinary level-by-level local reduction.

Two-level tree fusion. We separately tested a smaller

$$4 \rightarrow 2 \rightarrow 1$$

fusion. The current two-level path required approximately

$$9.7019 \mu\text{s},$$

while the fused version required

$$9.8528 \mu\text{s}.$$

Thus the fused variant was approximately

$$1.6\%$$

slower in this benchmark.

These negative results suggest that on the tested Apple M1 scalar path, reducing memory traffic is not automatically beneficial when it lengthens the dependency chain around the already optimized field fold.

4.12 Final implementation

The final evaluator used for the headline benchmark combines:

1. coarse-grained subtree parallelism;
2. block size $B = 4096$;
3. specialized Goldilocks multiplication reduction;
4. fused kernel-fold reduction;
5. unchecked inner-level indexing with explicit safety invariants;
6. direct first-level reduction into a half-size scratch buffer;
7. Rayon worker-local scratch reuse;
8. serial completion of the small top tree.

All of these optimizations preserve the exact fold graph derived in 3. They change only how the arithmetic is implemented and scheduled.

5 Benchmark Methodology

The purpose of the benchmark methodology is to isolate the effect of polynomial representation and implementation strategy while avoiding artificial disadvantages for competing representations. All reported timings are wall-clock measurements of actual Rust implementations.

5.1 Benchmark platform

The headline experiments were run on an Apple M1 system. The repository records the exact compiler, operating-system, processor, memory, Git revision, and thread-environment metadata associated with the benchmark in `paper/benchmark-environment.txt`.

This machine-generated metadata file is committed with the paper so that the software environment associated with the measurements can be inspected independently of the prose description.

The benchmark is compiled in Rust release mode. The release profile uses one code-generation unit, fat link-time optimization, and abort-on-panic semantics:

$$\text{codegen-units} = 1, \quad \text{lto} = \text{fat}, \quad \text{panic} = \text{abort}.$$

Where assembly inspection was required during optimization, compilation was performed for the native target CPU. The performance results reported in the benchmark tables are obtained from the release benchmark binaries described by the repository configuration.

5.2 Field

All compared polynomial representations use the same Goldilocks field implementation:

$$\mathbb{F}_p, \quad p = 2^{64} - 2^{32} + 1.$$

Using a common field backend is essential. Comparing one basis with a specialized field implementation against another basis with generic modular arithmetic would measure field engineering rather than the representation itself.

The final Boolean-kernel implementation uses the optimized Goldilocks arithmetic described in 4. Final cross-basis numbers must therefore be collected after the same arithmetic backend is active for every representation participating in the comparison.

5.3 Input sizes

The principal scaling experiment uses

$$N \in \{2^{10}, 2^{12}, 2^{14}, 2^{16}, 2^{18}, 2^{20}\}.$$

The headline comparison focuses on

$$N = 2^{20} = 1,048,576.$$

Power-of-two sizes are natural for the Boolean-kernel tree and also permit a genuine power-of-two roots-of-unity Lagrange domain in the Goldilocks field.

5.4 Input generation

For reproducibility, the monomial coefficient vector used by the main Criterion benchmark is deterministic. For

$$0 \leq i < N,$$

the implementation constructs

$$a_i = 17i + 3$$

as a canonical field element.

The Boolean-kernel coefficient vector is obtained from the same polynomial using the monomial-to-kernel conversion implemented in the repository. That conversion is performed outside the timed evaluation loop.

The evaluation point used by the main benchmark is

$$x = 19.$$

These deterministic values are chosen for reproducibility rather than for special algebraic structure. Correctness tests separately exercise the representation identities and optimized field arithmetic.

5.5 Measured operation

The principal operation is:

Given a polynomial already stored in a specified representation and an arbitrary field point x , compute $P(x)$.

The timed region therefore includes the work required to evaluate from that representation but excludes conversion from another representation.

This distinction is intentional. The benchmark asks how expensive single-point evaluation is once a prover already owns the polynomial in the representation being measured. End-to-end systems may need conversion, and those costs must be accounted for separately when evaluating a complete prover pipeline.

5.6 Monomial baselines

Two monomial baselines are retained.

The first is ordinary serial Horner evaluation. It is useful because it is the standard direct method and exposes the dependency chain inherent in that formulation.

The second is a blocked parallel evaluator. If the block size is B , write

$$P(X) = \sum_j X^{jB} P_j(X),$$

where each block polynomial P_j has degree less than B . The values $P_j(x)$ can be evaluated independently and combined afterward.

The paper uses this blocked implementation as the fair parallel monomial baseline. We do not claim a kernel speedup by comparing a parallel kernel implementation only against an artificially serialized competitor.

For the headline experiments the monomial block size is

$$B = 4096.$$

5.7 Boolean-kernel configuration

The final Boolean-kernel evaluator uses the implementation described in 4 with local subtree size

$$B = 4096.$$

The block-size study showed no reproducible advantage large enough to justify replacing this setting with 8192, so 4096 is retained as the conservative default.

Rayon provides the CPU worker pool. Unless a benchmark explicitly studies thread scaling, the default Rayon worker count for the machine is used. Thread-scaling experiments, when reported, must state `RAYON_NUM_THREADS` explicitly.

5.8 Criterion configuration

The principal Criterion benchmark uses:

- sample size: 50;
- warm-up time: 5 seconds;
- measurement time: 10 seconds.

For close block-size comparisons, a longer controlled benchmark was used:

- sample size: 100;
- warm-up time: 10 seconds;
- measurement time: 20 seconds.

We report the Criterion central estimate together with the measurement interval when discussing individual optimization steps. Outliers reported by Criterion are not silently removed from the benchmark configuration.

Because sub-millisecond timings on a laptop-class processor can move with thermal state and background activity, differences at approximately the one-percent level are treated cautiously and are not promoted to algorithmic claims without controlled reruns.

5.9 Correctness checks

Performance measurements are accepted only after correctness tests pass.

The repository checks, among other properties:

1. direct Boolean-kernel evaluation agrees with the tree evaluator;
2. monomial-to-kernel conversion preserves the represented polynomial;
3. parallel kernel implementations agree with the serial reference;
4. optimized Goldilocks multiplication agrees with the reference multiplication;
5. fused kernel folds agree with the unfused mathematical expression;
6. Lagrange evaluation reconstructs the same polynomial value as the monomial representation on test instances.

Optimization variants are therefore compared only after establishing that they compute the same field value.

5.10 Treatment of setup and precomputation

Setup is excluded from a timed loop only when it is reusable in the corresponding representation-level workload.

For example, converting monomial coefficients into Boolean-kernel coefficients is not charged to the kernel single-point evaluator because the benchmark assumes that the polynomial is already represented in that basis. Likewise, construction of a reusable roots-of-unity domain should not be charged repeatedly to every Lagrange query if the same domain is reused by the prover.

Conversely, temporary work intrinsic to one evaluation belongs in the timed path unless it is explicitly identified as reusable.

All exclusions must therefore be interpreted as representation assumptions, not as claims that conversion or preprocessing is free.

5.11 Lagrange comparison policy

The repository contains both serial and parallel arbitrary-point Lagrange evaluation over a roots-of-unity domain using batch inversion.

However, the numerical Lagrange results are not frozen at this stage of the paper. The Boolean-kernel arithmetic backend changed substantially during optimization, so an old Lagrange measurement collected under an earlier field implementation is not a valid final comparison.

Before a Lagrange number is placed in the final results table, the Lagrange implementation will be rerun using the same final field backend, machine, compiler profile, and Criterion methodology.

This prevents the paper from combining timings collected under materially different arithmetic implementations.

5.12 Fairness rules

The experimental comparison follows the following rules:

1. the same field implementation is used by all bases in a final comparison;
2. every method receives the same polynomial size N ;
3. all timings used in a direct ratio are collected on the same benchmark machine;
4. serial and parallel algorithms are labeled separately;
5. legitimate parallelism is allowed for competing bases;
6. representation conversion is reported separately from evaluation;
7. reusable preprocessing is excluded only when the corresponding workflow can genuinely reuse it;
8. all optimized implementations must agree with a correctness reference before their timings are accepted;
9. results within the observed noise floor are not treated as meaningful improvements.

These rules are designed to make the benchmark a comparison of useful algorithms rather than of deliberately favorable implementation choices.

6 Experimental Results

This section reports the final cross-basis benchmark collected in a single run using the same machine, compiler configuration, Goldilocks backend, and Criterion methodology for every representation.

The final comparison includes:

1. the optimized Boolean-kernel evaluator;
2. the blocked parallel monomial evaluator with block size 4096;
3. serial Horner evaluation;
4. parallel roots-of-unity Lagrange evaluation using batch inversion;
5. serial roots-of-unity Lagrange evaluation using batch inversion.

For every input size, all five methods represent the same deterministic polynomial and are checked for equality at the evaluation point before timing.

6.1 Cross-basis scaling

1 reports the Criterion central estimates for all six tested sizes.

N	Kernel	Monomial par.	Horner	Lagrange par.	Lagrange ser.
2^{10}	$4.343 \mu\text{s}$	$8.613 \mu\text{s}$	$8.708 \mu\text{s}$	$109.710 \mu\text{s}$	$21.612 \mu\text{s}$
2^{12}	$16.975 \mu\text{s}$	$34.599 \mu\text{s}$	$35.491 \mu\text{s}$	$250.220 \mu\text{s}$	$84.269 \mu\text{s}$
2^{14}	$59.153 \mu\text{s}$	$69.174 \mu\text{s}$	$146.720 \mu\text{s}$	$439.930 \mu\text{s}$	$334.240 \mu\text{s}$
2^{16}	$91.116 \mu\text{s}$	$139.480 \mu\text{s}$	$558.910 \mu\text{s}$	1.2503 ms	1.3391 ms
2^{18}	$263.230 \mu\text{s}$	$380.910 \mu\text{s}$	2.2440 ms	4.5612 ms	5.3679 ms
2^{20}	1.0560 ms	1.4116 ms	9.0402 ms	17.1990 ms	21.3890 ms

Table 1: Final same-run arbitrary-point evaluation benchmark over the Goldilocks field. Lower is better.

The Boolean-kernel evaluator is the fastest method at every tested size in this final sweep.

6.2 Headline result at $N = 2^{20}$

For

$$N = 2^{20} = 1,048,576,$$

the optimized Boolean-kernel evaluator has Criterion interval

$$[1.0009, 1.1132] \text{ ms}$$

with central estimate

$$\boxed{1.0560 \text{ ms}}.$$

The corresponding throughput is approximately

$$\boxed{9.93 \times 10^8}$$

input coefficients per second.

The parallel monomial baseline has central estimate

$$1.4116 \text{ ms}.$$

Thus, in the final same-run comparison,

$$\frac{1.4116}{1.0560} = \boxed{1.337\times}.$$

Equivalently, the Boolean-kernel evaluator uses approximately

$$1 - \frac{1.0560}{1.4116} \approx \boxed{25.2\%}$$

less wall-clock time than the blocked parallel monomial evaluator.

Against serial Horner evaluation,

$$\frac{9.0402}{1.0560} = \boxed{8.56\times}.$$

6.3 Comparison with the Lagrange representation

At

$$N = 2^{20},$$

parallel arbitrary-point evaluation from the roots-of-unity Lagrange representation requires

$$17.199 \text{ ms},$$

while the serial version requires

$$21.389 \text{ ms}.$$

Relative to the optimized Boolean-kernel evaluator, these correspond to

$$\frac{17.199}{1.0560} = \boxed{16.29\times}$$

and

$$\frac{21.389}{1.0560} = \boxed{20.25\times},$$

respectively.

This result concerns *arbitrary-point evaluation from the stored representation*. It should not be interpreted as evidence that roots-of-unity evaluation form is generally inferior in a proof system. Evaluation form is highly advantageous for other workloads, including pointwise operations and FFT/NTT-based encoding. The comparison instead shows that an arbitrary off-domain evaluation has a very different cost profile in the three representations considered here.

6.4 Scaling relative to the kernel evaluator

The ratio between each competitor and the Boolean-kernel evaluator changes with N . For the fair parallel monomial baseline, the measured ratios are approximately

N	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
$T_{\text{mono,par}}/T_{\text{kernel}}$	1.98	2.04	1.17	1.53	1.45	1.34

The non-monotonic behavior reflects the interaction between subtree parallelism, Rayon scheduling, cache behavior, and the fixed block size. It is therefore more appropriate to report the measured curve than to infer a constant hardware-independent speedup factor.

For serial Horner evaluation, the ratio grows much more strongly with problem size:

$$2.00, 2.09, 2.48, 6.13, 8.52, 8.56.$$

This is consistent with the fact that the kernel tree exposes substantial intra-polynomial parallelism while Horner evaluation retains a long sequential dependency chain.

6.5 Best observed optimization run versus final comparison

During optimization of the Boolean-kernel evaluator alone, the best observed $N = 2^{20}$ run was

$$\boxed{892.57 \text{ } \mu\text{s}},$$

with interval

$$[884.25, 902.65] \text{ } \mu\text{s}.$$

The final multi-method sweep measured the same optimized kernel evaluator at

$$1.0560 \text{ ms}.$$

We deliberately use the latter number for the cross-basis headline ratio, because it was collected in the same benchmark campaign as all competing representations. The lower $892.57\ \mu\text{s}$ measurement is retained as an optimization result and evidence of run-to-run variability, but it is not mixed with competitor timings from a different run to construct a speedup claim.

This distinction prevents accidental cherry-picking.

6.6 Optimization ablation

The optimization campaign reduced the best observed subtree-parallel runtime approximately as follows:

$$1.2530 \rightarrow 1.2007 \rightarrow 1.1619 \rightarrow 1.0179 \rightarrow 0.93533 \rightarrow 0.89257 \text{ ms.}$$

The final value is approximately

$$28.8\%$$

below the earlier 1.2530-ms subtree baseline.

The major experimentally validated improvements were:

1. specialized Goldilocks multiplication reduction;
2. fused field-level kernel fold;
3. elimination of bounds checks in the hot level loop;
4. direct first-level reduction into half-size scratch;
5. worker-local scratch reuse.

The isolated microbenchmarks showed approximately:

Optimization	Before	After	Local improvement
Goldilocks multiplication	2.2372 ns	2.0559 ns	8.1%
Fused kernel fold	4.0591 ns	3.3585 ns	17.3%
Unchecked level loop	6.9671 μs	6.3575 μs	8.75%
Direct first level	6.9031 μs	6.1571 μs	10.8%

These percentages describe the affected primitive or stage and should not be added together to predict whole-evaluator speedup.

6.7 Negative results

Two-level and three-level tree fusion were both rejected. In particular, the two-level experiment gave

$$9.7019\ \mu\text{s}$$

for the ordinary two-level path and

$$9.8528\ \mu\text{s}$$

for the fused path.

Thus the version with fewer intermediate memory accesses was approximately 1.6% slower.

This result illustrates an important implementation constraint: reducing memory traffic does not necessarily improve scalar field code when doing so increases dependency depth and register pressure.

6.8 Interpretation

The final results establish a narrow but concrete claim:

For arbitrary single-point evaluation of a degree- $< N$ univariate polynomial already stored in the tested representation, the optimized Boolean-kernel evaluator is faster than the monomial and roots-of-unity Lagrange implementations in our Rust benchmark over Goldilocks on the tested Apple M1 machine.

At $N = 2^{20}$, the conservative same-run comparison gives a

$$1.337\times$$

advantage over the parallel monomial baseline and a

$$25.2\%$$

reduction in wall-clock evaluation time.

The experiment does not yet establish an end-to-end prover speedup. That requires measuring basis conversion, encoding, commitment, FRI-related operations, hashing, and all other protocol costs in an integrated system.

7 Relevance to Proof Systems and Limitations

The experiments in this paper isolate arbitrary single-point evaluation of a univariate polynomial already stored in a chosen representation. This section explains where such a primitive appears in existing proof-system architectures, which parts of the optimization can transfer directly, and which conclusions would require end-to-end integration experiments.

7.1 Where arbitrary-point evaluations appear

Transparent proof systems based on Reed–Solomon proximity testing commonly move between polynomial coefficients and evaluations on structured domains. They also evaluate committed polynomials at verifier-derived challenge points that need not lie on the original commitment domain.

RISC Zero provides a concrete public example of this pattern. Its STARK documentation describes conversion of trace columns to polynomial coefficients with an inverse NTT, Reed–Solomon evaluation on an expanded domain, DEEP-ALI checks at an out-of-domain point z , and a subsequent FRI low-degree argument [RIS26e, RIS26c].

In the DEEP-ALI stage, the prover supplies evaluations of trace polynomials at points of the form

$$P_i(\omega^n z),$$

where z is a verifier challenge and the shifts depend on the constraint system [RIS26c]. This is the class of operation for which an efficient arbitrary-point evaluator can potentially matter.

The relevance is nevertheless conditional on representation. Existing STARK implementations are typically organized around evaluation-domain vectors because FFT/NTT-based encoding, pointwise constraint operations, Merkle commitments, and FRI naturally consume such data. Replacing only one evaluation routine without considering the representation of the surrounding pipeline may therefore introduce conversion costs that dominate the local gain.

7.2 Relation to FRI

FRI proves that a committed vector is close to evaluations of a low-degree polynomial. Its commit phase recursively folds evaluations into smaller commitments, while the query phase opens a small number of positions from the committed layers [RIS26a].

The Boolean-kernel fold studied in this paper,

$$F_t(a, b) = t(a + b) + b,$$

should not be identified with a standard FRI fold merely because both are binary reductions. They act on different representations and satisfy different protocol roles.

The present result instead motivates the following systems question:

Can a prover retain a Boolean-kernel representation across enough of the polynomial pipeline that the fast arbitrary-point evaluator is useful without paying an offsetting conversion cost before FRI or commitment?

Answering this requires protocol-level design and benchmarking. The current paper establishes the performance of the local representation primitive; it does not establish a new FRI protocol.

7.3 Transferable low-level optimizations

Some optimizations presented in 4 are not specific to the Boolean-kernel basis.

The specialized Goldilocks reduction exploits only

$$p = 2^{64} - 2^{32} + 1.$$

It can therefore benefit other Goldilocks workloads when the same arithmetic pattern occurs.

This is relevant because Goldilocks has already been used specifically to obtain efficient CPU arithmetic in FRI-based proof systems. Plonky2, for example, was designed around FRI and the Goldilocks field precisely because the modulus permits efficient 64-bit arithmetic on modern processors [Pol22].

Likewise, the experiments on allocation reuse, worker-local scratch space, bounds-check elimination, and task granularity are general implementation lessons for prover kernels. Their exact gains are workload-dependent, but they do not rely on a change to the cryptographic protocol.

By contrast, the speedup arising from the Boolean-kernel representation itself transfers only if the relevant polynomial is actually available in that representation.

7.4 Effect on an existing prover

Suppose an existing prover currently represents a polynomial by monomial coefficients or by evaluations on a roots-of-unity domain.

There are three distinct integration cases.

Case 1: local arithmetic replacement only. If the prover retains its existing representation and proof format but adopts only the Goldilocks arithmetic optimizations, then the cryptographic protocol is unchanged. Any speedup is limited to the affected arithmetic kernels.

Case 2: internal representation change with identical proof interface. If the prover internally stores some polynomials in Boolean-kernel form but converts back before commitment or proof serialization, then an external verifier need not change. The benefit is positive only if

$$T_{\text{saved evaluations}} > T_{\text{basis conversions}} + T_{\text{integration overhead}}.$$

Case 3: protocol-visible representation change. If commitments, openings, FRI layers, transcript messages, or verifier equations themselves change, then the proof format changes. Existing verifiers cannot automatically accept the new proof format.

This distinction is important when discussing systems that already have deployed verification infrastructure.

7.5 On-chain verification

A prover optimization does not by itself reduce on-chain verification gas.

For example, RISC Zero publishes Solidity verifier interfaces for receipts and Ethereum integration contracts [RIS26b]. These contracts consume a particular proof or receipt format.

If a faster prover produces exactly the same proof bytes and verification relation, then:

prover time may decrease while on-chain verification remains unchanged.

In this case no smart-contract change is required, but gas usage is not expected to improve merely because the prover computed the proof faster.

If instead the Boolean-kernel representation changes the proof object or verification equations, then deployed contracts would need a compatible new verifier implementation, a routing/versioning mechanism, or a migration path. Existing RISC Zero Ethereum infrastructure, for example, uses verifier selectors and routing between verifier implementations, illustrating that verifier parameters and proof formats are versioned objects [RIS26d].

Accordingly, the present benchmark should be viewed primarily as a *prover-side* result.

7.6 When the benchmark is likely to matter

The representation is most promising in workloads satisfying several of the following conditions:

1. large polynomials are evaluated repeatedly at verifier-derived off-domain points;
2. the polynomial can remain in Boolean-kernel representation for more than one operation;
3. basis conversion can be amortized or eliminated;
4. the prover is CPU-bound in field arithmetic or memory movement rather than dominated by hashing or external I/O;
5. the target field admits efficient realization of the fold arithmetic.

Conversely, the result is less likely to affect workloads in which the polynomial is used almost exclusively through evaluations on one fixed FFT domain or where a single arbitrary-point evaluation is surrounded by much larger hashing, commitment, or encoding costs.

7.7 Representation conversion

The monomial-to-kernel conversion described in 2 is a Boolean Möbius transform and costs

$$O(N \log N)$$

field operations.

This cost is not included in the single-point evaluation timing because the benchmark assumes the input is already represented in the basis being measured.

For a one-off query, conversion can easily outweigh the evaluation gain. The basis is therefore most interesting when either:

1. the witness or polynomial is generated directly in kernel coefficients;

2. several subsequent operations consume the kernel representation;
3. conversion is fused with another prover stage;
4. or the representation replaces another transform already present in the prover.

An end-to-end evaluation must account for the full representation lifetime, not only the isolated query.

7.8 Limitations of the present experiments

The current study has several explicit limitations.

Single processor family. The headline measurements are from an Apple M1 system. Cache sizes, instruction scheduling, multiplication latency, memory bandwidth, and the cost of Rayon scheduling differ on x86-64 servers and other ARM systems.

Single base field. The final measurements use Goldilocks. The Boolean-kernel algebra is field-independent, but the constant factors of the implementation are not. A field such as Mersenne-31 may produce a different balance between arithmetic and memory costs.

CPU implementation. No GPU, FPGA, or wide-SIMD implementation is evaluated in this paper.

Single-point evaluation focus. The paper does not yet benchmark all operations needed by a proof system. In particular, full-domain encoding, low-degree extension, commitment, basis conversion, and FRI integration are separate workloads.

No end-to-end prover measurement. The measured

$$1.337\times$$

same-run speed advantage over the parallel monomial baseline applies to the single-point evaluator only. It is not an end-to-end STARK or PCS speedup.

Run-to-run variability. The best kernel-only optimization run reached

$$892.57\,\mu\text{s},$$

while the final same-run cross-basis sweep measured

$$1.0560\,\text{ms}.$$

The paper therefore uses same-run measurements when constructing cross-basis ratios and reports the lower number only as a best observed implementation result.

Lagrange implementation scope. The roots-of-unity Lagrange benchmark uses the direct barycentric formula with batch inversion for an arbitrary point. It is not intended to replace FFT/NTT evaluation when the requested points form a structured domain.

7.9 Next system-level experiments

The benchmark naturally suggests the following next experiments:

1. measure monomial-to-kernel and kernel-to-monomial conversion alongside single-point evaluation;
2. measure repeated queries to determine the query count at which conversion amortizes;
3. integrate the evaluator into a DEEP-style off-domain evaluation stage of an existing STARK implementation;
4. benchmark a kernel-native route to FRI commitment and folding;
5. repeat the study on x86-64, Mersenne-31, and additional processor architectures;
6. measure the fraction of total prover time attributable to the operation before making an end-to-end performance claim.

The principal open engineering question is therefore no longer whether the Boolean-kernel representation supports a fast arbitrary-point evaluation. The experiments establish that it does. The remaining question is whether a complete proof-system pipeline can preserve the representation long enough for this local advantage to survive at system scale.

8 Conclusion

We studied arbitrary single-point evaluation of degree- $< N$ univariate polynomials represented in the Boolean-kernel basis

$$K_y(X) = \prod_{i=0}^{m-1} (X^{2^i} + y_i), \quad N = 2^m, \quad y \in \{0, 1\}^m.$$

The representation admits the local fold

$$F_t(a, b) = t(a + b) + b, \quad t = x^{2^i},$$

which evaluates the polynomial through a complete binary reduction tree. The tree contains exactly

$$N - 1$$

folds and therefore requires exactly

$$N - 1$$

field multiplications and

$$2(N - 1)$$

field additions, together with

$$m - 1$$

squarings to generate the level parameters.

On the implementation side, the final Rust evaluator combines coarse-grained subtree parallelism, specialized Goldilocks reduction, a fused kernel fold, bounds-check elimination in the hot loop, direct first-level reduction, and worker-local scratch reuse.

For

$$N = 2^{20},$$

the final same-run benchmark on the tested Apple M1 system measured

1.0560 ms

for the optimized Boolean-kernel evaluator and

1.4116 ms

for the blocked parallel monomial baseline. This corresponds to a

$1.337\times$

evaluation-speed advantage, or approximately

25.2%

lower wall-clock time for the isolated arbitrary-point evaluation primitive.

A separate kernel-only optimization run reached

$892.57\ \mu\text{s}$,

but we do not combine that number with competitor timings collected in a different run when computing cross-basis speedups.

The comparison with the roots-of-unity Lagrange representation shows an even larger difference for arbitrary off-domain evaluation. At $N = 2^{20}$, the measured parallel Lagrange evaluator required 17.199 ms. This does not imply that evaluation form is generally inferior in proof systems; rather, it emphasizes that representation choice is workload-dependent.

The central systems question is therefore not whether the Boolean-kernel basis supports fast single-point evaluation—the experiments establish that it does—but whether a prover can retain this representation across enough of its pipeline that conversion, encoding, commitment, and FRI-related costs do not erase the local advantage.

Future work should measure that representation lifetime directly inside a complete transparent proof system, including basis conversion, repeated off-domain queries, commitment, and FRI integration. Only such an end-to-end study can determine how much of the isolated evaluator speedup survives at prover scale.

References

- [BSBHR18] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018.
- [KT23] Tohru Kohrita and Patrick Towa. Zeromorph: Zero-knowledge multilinear-evaluation proofs from homomorphic univariate commitments. Cryptology ePrint Archive, Paper 2023/917, 2023.
- [Pol22] Polygon. Introducing plonky2. Polygon Technology Blog, 2022.
- [RIS26a] RISC Zero. About the fri protocol. RISC Zero documentation, 2026.
- [RIS26b] RISC Zero. Iriszeroverifier solidity interface. RISC Zero Ethereum repository, 2026.
- [RIS26c] RISC Zero. The risc zero stark protocol. RISC Zero documentation, 2026.
- [RIS26d] RISC Zero. Risczeroverifierrouter. RISC Zero Ethereum repository, 2026.
- [RIS26e] RISC Zero. Stark by hand. RISC Zero documentation, 2026.